
Module 4: Convolutional Neural Networks

Introduction to Deep Learning

Lecture 1: Fundamentals · Lecture 2: Architectures
Lecture 3: Advanced Training · Lecture 4: Applications

4 classes × 50 minutes

Lecture 1 — Fundamentals

- › Why CNNs? The spatial inductive bias
- › Images as tensors
- › Convolution, stride, padding
- › Pooling layers

Lecture 2 — Architectures

- › LeNet → AlexNet → VGGNet
- › GoogLeNet & Inception modules
- › ResNet & residual connections

Lecture 3 — Advanced Training

- › Batch Normalization
- › Activation functions: ReLU variants, Swish, GELU
- › Regularisation in CNNs
- › Data augmentation

Lecture 4 — Applications

- › Object Detection: R-CNN family, YOLO, SSD
- › Semantic Segmentation: FCN, U-Net
- › Transfer learning

Lecture 1

CNN Fundamentals: Image Processing, Convolution, Stride,
Padding & Pooling

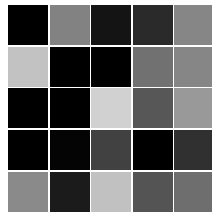
Why Not Plain MLPs for Images?

The Problem with Fully-Connected Layers

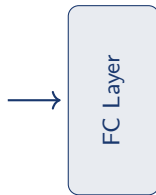
A 224×224 RGB image has $224 \times 224 \times 3 = 150,528$ input pixels.

A single hidden layer with 1024 units requires

$$150,528 \times 1024 \approx 154\text{M parameters!}$$



5×5 grayscale patch



154M params

Key problems

- **Parameter explosion** — intractable for high-res images
- **No spatial awareness** — pixel order is ignored
- **No translation invariance** — a shifted cat is a new pattern
- **Overfitting** — too many parameters, too little data

Inductive Biases of CNNs

Three ideas

1 Local connectivity

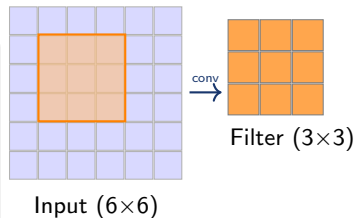
Each neuron connects to a small *receptive field*, not all pixels.

2 Weight sharing

The same filter is applied across all spatial locations.

3 Translation equivariance

If the input shifts, the feature map shifts by the same amount.



Weight sharing: the same 3×3 filter slides over the entire input

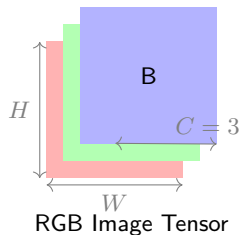
Result

A 3×3 filter on a 224×224 image uses only **9 weights** (+ bias) regardless of image size!

Representing Images as Tensors

Tensor Shape Conventions

- A single image: $(H \times W \times C)$
Height $\times$ Width $\times$ Channels
- A mini-batch of N images: $(N \times C \times H \times W)$
(PyTorch convention: NCHW)
- After a conv layer with K filters:
output shape = $(N \times K \times H' \times W')$



Pixel values

- Raw: integer in $[0, 255]$
- Normalised (typical): float in $[0, 1]$ or
zero-mean: $\frac{x - \mu}{\sigma}$ per channel

The Convolution Operation

Discrete 2D Convolution

$$(I * K)[i, j] = \sum_m \sum_n I[i + m, j + n] \cdot K[m, n]$$

- › I — input feature map (or image)
- › K — learnable kernel (filter)
- › Each position (i, j) in the output is a **dot product** between the kernel and a local patch

Note

In deep learning we technically perform *cross-correlation*, but call it convolution. Weights are learned — the distinction does not matter.

1	2	3	0	1
0	1	2	3	0
1	0	1	2	3
0	1	0	1	2
1	2	1	0	1

Input (5×5)



1	0	-1
1	0	-1
1	0	-1

Kernel (3×3)

Output (3×3)

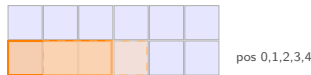
Stride

Definition

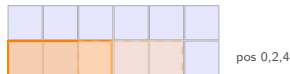
Stride s is the step size by which the filter moves across the input.

- › **Stride = 1**: filter moves one pixel at a time (default)
- › **Stride = 2**: filter jumps 2 pixels — output is roughly half the size
- › Larger stride $\Rightarrow$ smaller feature map $\Rightarrow$ faster computation

Stride = 1



Stride = 2



Output size formula (no padding)

$$W_{\text{out}} = \left\lfloor \frac{W_{\text{in}} - F}{s} \right\rfloor + 1$$

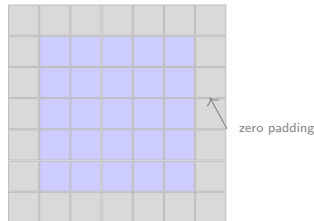
W_{in} = input width, F = filter size, s = stride

Padding

Why padding?

Without padding, every conv layer shrinks the spatial dimensions.

- Adds extra pixels (usually **zeros**) around the border
- **Valid padding** ($p = 0$): no padding, output shrinks
- **Same padding**: output = input size (for $s = 1$): $p = \lfloor F/2 \rfloor$
- Allows building **deep networks** without spatial collapse



5×5 input with $p = 1$ padding

General output size formula

$$W_{\text{out}} = \left\lfloor \frac{W_{\text{in}} + 2p - F}{s} \right\rfloor + 1$$

Multiple Channels and Filters

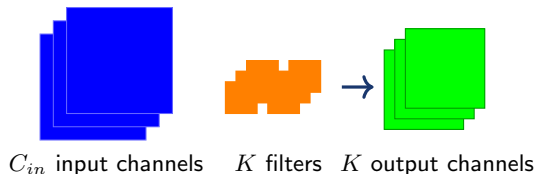
Multi-channel convolution

For input with C_{in} channels and K filters of size $F \times F$:

- › Each filter has shape $C_{in} \times F \times F$
- › Output has K channels (one per filter)
- › **Parameters per layer:** $K \times (C_{in} \times F^2 + 1)$

Example: First AlexNet conv layer

- › Input: $3 \times 224 \times 224$
- › 96 filters of 11×11 , stride 4
- › Params: $96 \times (3 \times 121 + 1) = 34,944$
- › Output: $96 \times 55 \times 55$



Pooling Layers

Purpose of pooling

- **Spatial downsampling** — reduce H & W
- Achieves approximate **translation invariance**
- Reduces computation and overfitting
- No learnable parameters

Common types

- **Max pooling**: take maximum in region
Most common; preserves dominant features
- **Average pooling**: take mean
Smoother; used in GoogLeNet
- **Global average pooling (GAP)**: collapse entire spatial dim to 1×1
Used before classifier head in modern CNNs

1	2	
2	3	
1	2	
3	1	

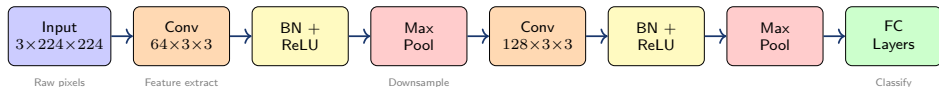
4×4 input

Max Pool 2 × 2

4	
3	

2×2 output

The Anatomy of a CNN



Repeated block

Conv → BN → ReLU → Pool

Spatial hierarchy

Early: edges & textures
Deep: parts & objects

Final layers

Flatten/GAP → FC → Softmax

Key Takeaways

- MLPs fail at images due to parameter explosion and lack of spatial awareness
- CNNs exploit **local connectivity**, **weight sharing**, and **translation equivariance**
- A convolution is a learnable, sliding dot-product
- **Stride** controls step size; **padding** controls output size
- **Max pooling** downsamples and adds invariance

Essential Formulas

$$W_{\text{out}} = \left\lfloor \frac{W_{\text{in}} + 2p - F}{s} \right\rfloor + 1$$

Parameters per conv layer:

$$K \times (C_{\text{in}} \times F^2 + 1)$$

Coming up in Lecture 2

How stacking these layers in clever ways led to the modern CNN revolution — from LeNet to ResNet.

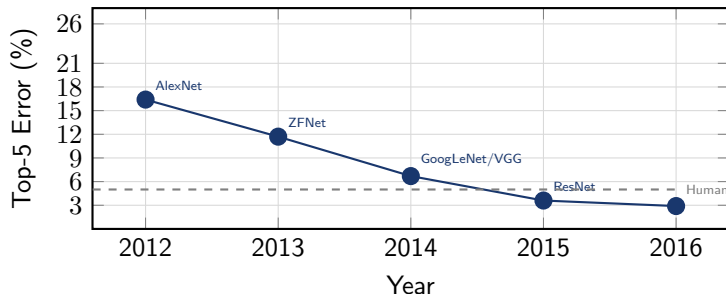
Lecture 2

CNN Architectures: LeNet, AlexNet, VGGNet, GoogLeNet,
ResNet

The ImageNet Challenge and the Architecture Race

ImageNet Large Scale Visual Recognition Challenge (ILSVRC)

- 1.2M training images, 1000 classes — the benchmark for CNN progress
- Top-5 error: model's top 5 predictions must include correct class



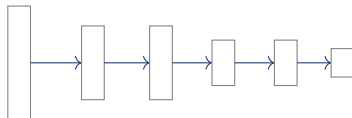
LeNet-5 (LeCun et al., 1998)

Architecture

Input 32×32 grayscale (MNIST)

- 1 Conv1: 6 filters, 5×5 , tanh $\rightarrow 28 \times 28 \times 6$
- 2 AvgPool: $2 \times 2 \rightarrow 14 \times 14 \times 6$
- 3 Conv2: 16 filters, 5×5 , tanh $\rightarrow 10 \times 10 \times 16$
- 4 AvgPool: $2 \times 2 \rightarrow 5 \times 5 \times 16$
- 5 Flatten $\rightarrow$ FC(120) $\rightarrow$ FC(84) $\rightarrow$ 10 classes

Total parameters: $\approx 60,000$



Layer stack (height = feature map size)

Historical significance

First practical CNN; proved CNNs work for digit recognition. Deployed in cheque reading systems at US banks.

AlexNet (Krizhevsky et al., 2012)

Key innovations

- › **ReLU activations** — faster training than sigmoid/tanh
- › **Dropout** (0.5) in FC layers — regularization
- › **Data augmentation** — random crops, flips
- › **GPU training** — trained on 2× GTX 580s (split across GPUs)
- › **Local Response Normalization (LRN)** — lateral inhibition
- › **Overlapping max-pooling**

Impact

Cut top-5 error from 26% (2011 winner) to **16.4%** — a stunning breakthrough that launched the deep learning era in vision.

AlexNet at a glance

Layer	Size	Filters	Params
Conv1	55×55	96	35K
Pool1	27×27	—	—
Conv2	27×27	256	614K
Pool2	13×13	—	—
Conv3	13×13	384	885K
Conv4	13×13	384	1.3M
Conv5	13×13	256	884K
Pool3	6×6	—	—
FC6	4096	—	38M
FC7	4096	—	17M
FC8	1000	—	4M
Total			≈60M

VGGNet (Simonyan & Zisserman, 2014)

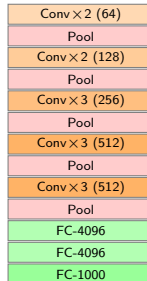
Core design philosophy

- Replace large filters (7×7 , 11×11) with **stacks of 3×3 filters**
- Two 3×3 convs have receptive field of 5×5 , but **fewer params** and **more non-linearity**
- Very deep, very simple: same pattern repeated
- Variants: VGG-11, VGG-13, **VGG-16**, **VGG-19**
- $\approx 138\text{M}$ parameters (mostly in FC layers)

Why 3×3 ?

Three 3×3 conv layers $\equiv$ one 7×7 receptive field
Parameters: $3 \times (3^2 C^2) = 27C^2$ vs $7^2 C^2 = 49C^2$
45% fewer parameters, more non-linearities!

VGG-16 Architecture



GoogLeNet & the Inception Module (Szegedy et al., 2014)

Problem with deep & wide networks

More layers + more filters = more parameters + more compute.

Can we be smarter about this?

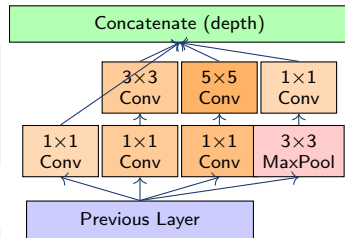
The Inception module

Apply **multiple filter sizes in parallel** at the same layer:

- 1×1 conv (dimensionality reduction)
- 3×3 conv
- 5×5 conv
- 3×3 max pool

Concatenate outputs along channel dim.

1×1 convolutions act as bottleneck to control channels.



Why residual connections help

- Gradient flows directly back through the shortcut — **gradient highway**
- Easier to learn identity mapping than arbitrary mapping
- Enables training of **very deep** networks (50, 101, 152 layers)

Bottleneck block (ResNet-50+)

1×1 (reduce) $\rightarrow 3 \times 3 \rightarrow 1 \times 1$ (expand)

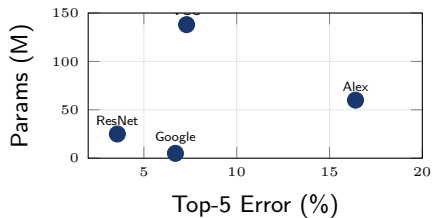
Reduces FLOPs while maintaining representational power

Result

ResNet-152: **3.57%** top-5 error on ImageNet — **better than human!**

Architecture Comparison

Model	Depth	Params
LeNet-5	7	60K
AlexNet	8	60M
VGG-16	16	138M
GoogLeNet	22	5M
ResNet-50	50	25M
ResNet-152	152	60M



Architecture Evolution

- **LeNet**: Proof of concept for CNNs
- **AlexNet**: Deep learning revolution; ReLU, Dropout, GPU training
- **VGGNet**: Simplicity through deep 3×3 stacks
- **GoogLeNet**: Multi-scale Inception modules, 1×1 bottlenecks
- **ResNet**: Residual connections solve vanishing gradient — enables *very* deep networks

Design principles to remember

- 1 Depth helps (if you can train it)
- 2 Small filters are better than large ones
- 3 Bottleneck 1×1 convs save compute
- 4 Skip connections are crucial for very deep nets

Up next: Lecture 3

The training tricks that make deep CNNs actually work

Lecture 3

Advanced Training: Batch Normalization & Improved
Activation Functions

Why Training Deep CNNs Is Difficult

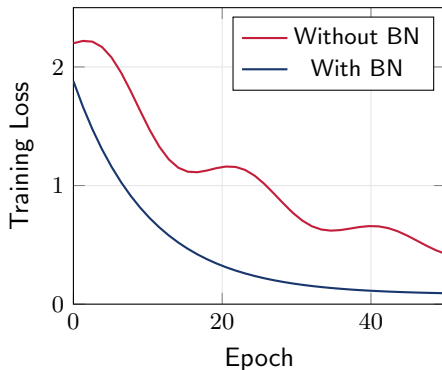
Core problem: Internal Covariate Shift

As parameters update during training, the distribution of *inputs to each layer* changes.

Later layers must continuously adapt to new distributions — **slow convergence**.

Consequences

- Must use very small learning rates
- Requires careful weight initialization
- Saturating activations (sigmoid, tanh) cause vanishing gradients
- Hard to train beyond 10 layers without tricks



Batch Normalization (Ioffe & Szegedy, 2015)

The idea

Normalize activations within each mini-batch, then **rescale and shift** with learnable parameters.

Algorithm (for a mini-batch

$$\mathcal{B} = \{x_1, \dots, x_m\})$$

- ➊ Compute batch mean: $\mu_{\mathcal{B}} = \frac{1}{m} \sum_i x_i$
- ➋ Compute batch variance: $\sigma_{\mathcal{B}}^2 = \frac{1}{m} \sum_i (x_i - \mu_{\mathcal{B}})^2$
- ➌ Normalize: $\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}$
- ➍ Scale & shift: $y_i = \gamma \hat{x}_i + \beta$

γ, β are **learned** per feature; ϵ for numerical stability.

Where to place BN?

- Canonical: **after** conv/FC, **before** activation
- Modern practice often: conv $\rightarrow$ BN $\rightarrow$ ReLU
- Pre-activation variant: BN $\rightarrow$ ReLU $\rightarrow$ conv (He et al., 2016)

Benefits

- Allows **much higher learning rates**
- Acts as a **regulariser** (reduces need for Dropout)
- **Faster convergence** (10 $\times$ or more)
- Reduces sensitivity to weight init

Batch Normalization: Training vs. Inference

Training mode

- Use **batch statistics** μ_B, σ_B^2 for normalization
- Maintain running averages of μ and σ^2 using exponential moving average
- Batch size must be > 1 (typically ≥ 16 for stable estimates)

Common mistake

Forgetting to set `model.eval()` in PyTorch switches BN from batch stats to population stats!

`model.train()` → batch statistics

`model.eval()` → population statistics

Inference mode

- **Cannot** use batch statistics (may have batch size = 1)
- Instead use **population statistics** accumulated during training:

$$\hat{x} = \frac{x - \mathbb{E}[x]}{\sqrt{\text{Var}[x] + \epsilon}}$$

BN Variants

- **Layer Norm** — normalize across features (used in Transformers)
- **Instance Norm** — normalize each sample (style transfer)
- **Group Norm** — compromise, small-batch friendly

Activation Functions: From Sigmoid to Modern

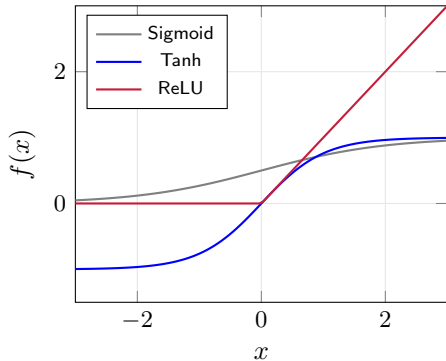
Classic activations and their problems

- › **Sigmoid** $\sigma(x) = \frac{1}{1+e^{-x}}$:
Saturates $\Rightarrow$ vanishing gradient; not zero-centered
- › **Tanh** $\tanh(x)$:
Zero-centered, but still saturates

ReLU (Nair & Hinton, 2010)

$$\text{ReLU}(x) = \max(0, x)$$

- › **Solves vanishing gradient** for $x > 0$
- › Computationally trivial
- › Creates **sparse activations**
- › Problem: **dying ReLU** — neurons stuck at 0



ReLU Variants

Leaky ReLU (Maas et al., 2013)

$$\text{LReLU}(x) = \begin{cases} x & x > 0 \\ \alpha x & x \leq 0 \end{cases}$$

Fixes dying ReLU: small negative slope $\alpha \approx 0.01$.

ELU (Clevert et al., 2015)

$$\text{ELU}(x) = \begin{cases} x & x > 0 \\ \alpha(e^x - 1) & x \leq 0 \end{cases}$$

Smooth negative saturation; zero-centered outputs.

Swish (Ramachandran et al., 2017)

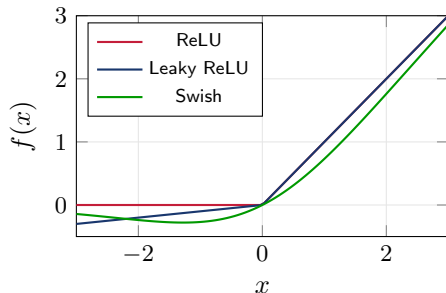
$$\text{Swish}(x) = x \cdot \sigma(x)$$

GELU (Hendrycks & Gimpel, 2016)

$$\text{GELU}(x) = x \cdot \Phi(x)$$

Φ = Gaussian CDF; stochastic regularization interpretation.

Default in BERT, GPT, ViT.



Regularisation in CNNs

Dropout (Srivastava et al., 2014)

- Randomly zero out units with probability p during training
- Forces redundant representations; implicit ensemble
- Commonly applied to **FC layers** ($p = 0.5$)
- Rarely applied in conv layers (or small p)
- At inference: scale by $(1 - p)$ or use **inverted dropout**

Weight Decay (L_2 Regularization)

Add $\frac{\lambda}{2} \|\mathbf{W}\|^2$ to the loss.

Penalizes large weights; typical $\lambda = 10^{-4}$.

Data Augmentation

Artificially increase dataset size by transforming inputs:

- **Geometric**: random crop, flip, rotation, scale
- **Photometric**: brightness, contrast, color jitter
- **CutOut**: zero out random square patches
- **Mixup**: blend two training images & labels
- **CutMix**: paste patches from one image into another
- **RandAugment**: automated augmentation policy

Rule of thumb

Weight Initialization

Why it matters

Poor initialization $\Rightarrow$ gradients vanish or explode before training even begins.

Xavier / Glorot Initialization

For **tanh/sigmoid**: draw weights from

$$\mathcal{U}\left(-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}, \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}\right)$$

Goal: preserve variance of activations across layers.

He Initialization

For **ReLU** (He et al., 2015):

$$\mathcal{U}\left(-\sqrt{2}, \sqrt{2}\right)$$

PyTorch defaults

`nn.Conv2d` uses Kaiming Uniform (He) by default.

```
nn.init.kaiming_normal_(m.weight)  
nn.init.xavier_uniform_(m.weight)
```

Key principle

With Batch Norm, initialization matters less — BN re-normalizes each layer's input regardless. Still best practice to use He/Xavier as starting point.

Batch Normalization

- › Reduces internal covariate shift
- › Allows large learning rates, faster convergence
- › Acts as regularizer
- › Different behavior at train vs. inference time
- › Learnable γ, β ; running stats for inference

Activation Functions

- › ReLU: simple, effective, standard baseline
- › Leaky ReLU: fixes dying neurons
- › Swish, GELU: smooth, state-of-the-art

Regularization Toolkit

- › Dropout (especially for FC layers)
- › Weight decay (L_2)
- › Data augmentation (often most impactful)
- › BN itself acts as regularizer

Modern best practice recipe

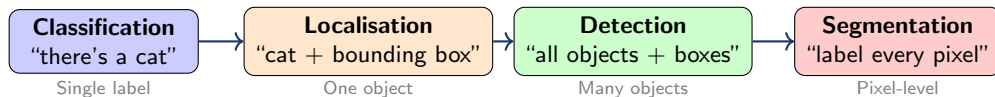
Conv $\rightarrow$ BN $\rightarrow$ ReLU/GELU $\rightarrow$ Dropout (optional)
+ He init + AdamW optimizer + data augmentation

Lecture 4

Applications: Object Detection & Semantic Segmentation

From Classification to Spatial Tasks

Hierarchy of visual recognition tasks



Detection output

For each object: class label + **bounding box**
(x, y, w, h)

Key challenge: **variable number of objects**

Segmentation output

A **dense prediction**: class label for every pixel

Semantic: all cats = same label; **Instance**: each cat separate

Region-based CNNs: R-CNN (Girshick et al., 2014)

R-CNN pipeline

- 1 **Selective search** generates ~ 2000 region proposals
- 2 Warp each proposal to fixed size (227×227)
- 3 Pass each through CNN to extract features
- 4 Classify with SVM per class
- 5 Refine bounding boxes with regression

Problem: Painfully slow

2000 CNN forward passes per image.

Training: 84 hours; Test: 47 seconds/image

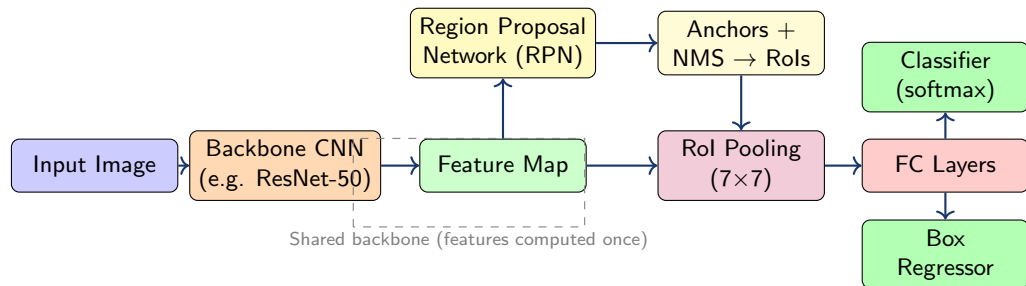
Fast R-CNN (Girshick, 2015)

- Run CNN **once** on full image
- Project proposals onto feature map
- **RoI Pooling**: pool each proposal to fixed size
- Single network for classification + box regression
- **Test: 2 sec/image (25× faster)**

Faster R-CNN (Ren et al., 2015)

- Replace selective search with **Region Proposal Network (RPN)**
- RPN shares features with detection head
- **Test: ~ 0.2 sec/image: end-to-end**

Faster R-CNN: Architecture Deep Dive



Anchors

Pre-defined boxes at multiple **scales** and **aspect ratios** at each position.

RPN predicts: *object?* + box refinement for each anchor.

Non-Maximum Suppression (NMS)

Remove duplicate detections: keep highest-scoring box, suppress overlapping boxes with $\text{IoU} > \text{threshold}$.

YOLO: You Only Look Once (Redmon et al., 2016)

Core idea: single-shot detection

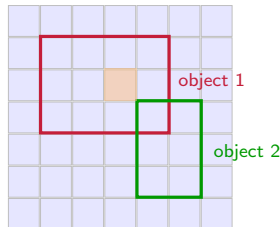
- › Divide image into $S \times S$ grid (e.g. 7×7)
- › Each cell predicts B bounding boxes + confidence scores + C class probabilities
- › **One forward pass through the network = detections**
- › **Real-time:** YOLOv1 runs at 45 fps (vs Faster R-CNN: ≈ 5 fps)

Loss function (multi-task)

$$L = L_{\text{coord}} + L_{\text{conf}} + L_{\text{class}}$$

Weighted sum of localization, confidence, and classification losses.

$\lambda_{\text{coord}} = 5$, $\lambda_{\text{noobj}} = 0.5$ balances terms.



7×7 Grid

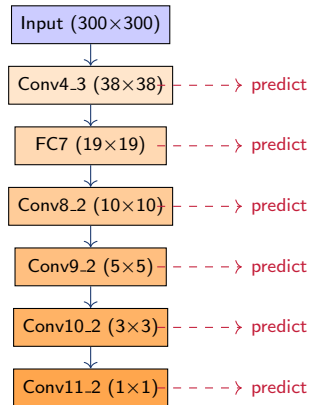
YOLO versions

YOLOv1 $\rightarrow$ v2 (anchors) $\rightarrow$ v3 (multi-scale) $\rightarrow$ v4/v5/v7/v8 (many improvements, state of the art in speed/accuracy)

SSD: Single Shot MultiBox Detector (Liu et al., 2016)

SSD key ideas

- Use **feature maps from multiple scales** in the backbone
- At each scale, use anchors of appropriate sizes
- **Small objects**: detected at high-res early feature maps
- **Large objects**: detected at low-res deep feature maps
- No region proposals — single shot, like YOLO
- Faster than Faster R-CNN; more accurate than YOLOv1 (at the time)



Multi-scale predictions

SSD300 on PASCAL VOC

59 fps at 74.3% mAP — best speed/accuracy tradeoff at release.

Object Detection Methods: Comparison

Method	Year	mAP	Speed	Stage	Key idea
R-CNN	2014	58.5	47s/img	2-stage	Region proposals + CNN
Fast R-CNN	2015	70.0	2s/img	2-stage	RoI Pooling
Faster R-CNN	2015	73.2	0.2s/img	2-stage	RPN (end-to-end)
YOLO v1	2016	63.4	22ms/img	1-stage	Grid + single forward pass
SSD300	2016	74.3	17ms/img	1-stage	Multi-scale anchors
YOLOv3	2018	82.1	29ms/img	1-stage	Multi-scale + Darknet53

mAP on PASCAL VOC 2007 / COCO (approximate — values vary by benchmark)

2-stage detectors

Higher accuracy; slower. Best when precision is critical (medical, autonomous driving offline).

1-stage detectors

Faster; great for real-time applications (robotics, video surveillance, edge devices).

Semantic Segmentation: Dense Prediction

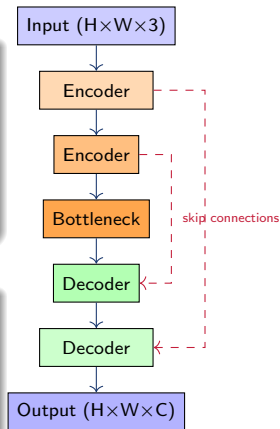
The challenge

Classify **every pixel** in the image.

- Output: $H \times W \times C$ probability map (C classes)
- Need **spatial detail** (fine-grained) + **context** (global)
- Pooling loses spatial information — recovery?

Fully Convolutional Networks (FCN)

- Replace all FC layers with conv layers $\Rightarrow$ arbitrary input size
- Use **transposed convolutions** (deconv) to upsample back to input resolution
- Add **skip connections** from encoder to preserve detail

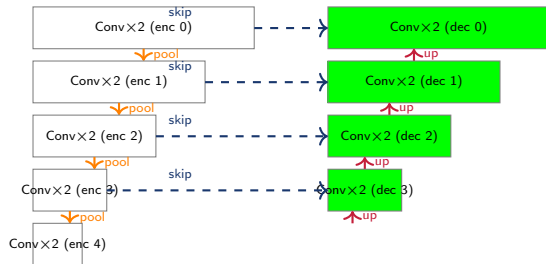


U-Net: For Dense Prediction (Ronneberger et al., 2015)

Architecture

Symmetric encoder-decoder (**U-shape**):

- **Contracting path** (encoder):
repeated
 $[\text{Conv} \rightarrow \text{BN} \rightarrow \text{ReLU}] \times 2 \rightarrow \text{MaxPool}$
Doubles channels at each level
- **Expansive path** (decoder):
Transposed conv (upsample) + **skip connection** (concatenate encoder feature map) + conv blocks
- **Bottleneck**: deepest layer, most channels, smallest spatial dim
- Final: 1×1 conv $\rightarrow$ pixel-wise classification



U-Net: Key Properties and Applications

Why U-Net works so well

- Skip connections pass **high-frequency spatial detail** directly to decoder
- Encoder captures **semantic context**; decoder reconstructs **spatial layout**
- Works with **very small datasets** (100s of images) — critical for medical imaging
- Data augmentation with elastic deformations compensates for limited training data
- Fully convolutional: works at any resolution

Applications

- **Medical image segmentation**: tumour delineation, cell segmentation, organ boundaries
- **Satellite / aerial imaging**: road extraction, building detection
- **Autonomous driving**: drivable surface segmentation
- **Industrial inspection**: defect detection

Impact

U-Net introduced in 2015 remains the **gold standard** for biomedical image segmentation; cited > 70,000 times.

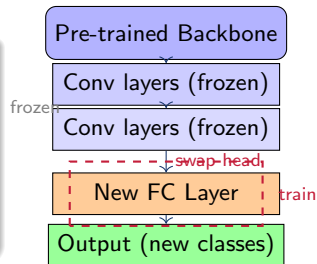
Transfer Learning with CNNs

Why transfer learning?

- Training from scratch requires millions of labeled images + days of compute
- CNN features generalize: low-level features (edges, textures) are universal
- Pre-trained models encode rich visual knowledge

Strategies

- **Feature extraction**: freeze backbone, train only new head
For small dataset + similar domain
- **Fine-tuning**: unfreeze some/all backbone layers with small LR
For larger dataset or different domain
- **Full fine-tuning**: treat pre-trained weights as initialization
For very large target dataset



Module 4 Summary: The Big Picture

Lecture 1: Fundamentals

Local connectivity, weight sharing, convolution, stride, padding, pooling

Lecture 2: Architectures

LeNet → AlexNet → VGGNet → GoogLeNet → ResNet

Key trend: *deeper, more efficient, skip connections*

Lecture 3: Training

Batch Norm, ReLU variants (Swish, GELU), Dropout, augmentation

Key trend: *training stability = architecture quality*

Lecture 4: Applications

Detection: R-CNN → Faster R-CNN → YOLO → SSD

Segmentation: FCN → U-Net

Key trend: *from global labels to dense, real-time prediction*

Core insight of CNNs

The same simple idea — **learn local, shared filters** — scales from 32×32 digit recognition to real-time detection of 80+ object classes in 4K video. The **architecture and training** choices determine how far it scales.

Papers

- › **LeNet**: LeCun et al. (1998) — *Gradient-Based Learning...*
- › **AlexNet**: Krizhevsky et al. (NeurIPS 2012)
- › **VGG**: Simonyan & Zisserman (ICLR 2015)
- › **GoogLeNet**: Szegedy et al. (CVPR 2015)
- › **ResNet**: He et al. (CVPR 2016)
- › **BatchNorm**: Ioffe & Szegedy (ICML 2015)
- › **Dropout**: Srivastava et al. (JMLR 2014)
- › **Faster R-CNN**: Ren et al. (NeurIPS 2015)
- › **YOLO**: Redmon et al. (CVPR 2016)
- › **SSD**: Liu et al. (ECCV 2016)
- › **U-Net**: Ronneberger et al. (MICCAI 2015)